In the context of development or testing, these transient logs Kubernetes offers are rather useful, as you can tell from the following example (note that the output has been edited):

With kubectl logs deployment, we ask for the collective logs from all pods under the control of the deployment. In our example, the deployment is called frontend, and we specifically are interested in the logs of the container ho11y. Finally, we limit the output (displayed log lines) with head.

```
$ kubectl logs deployment/frontend ho11y -f | head -4
{"level":"info","msg":"Using http://d...0","time":"2022-03-25T11:35:51Z"}
{"level":"info","msg":"Using http://d...1","time":"2022-03-25T11:35:51Z"}
{"level":"info","msg":"Using adot:4317","time":"2022-03-25T11:35:51Z"}
{"level":"info","msg":"Init instr... done","time":"2022-03-25T11:35:51Z"}
```

The workload or application-level logs from the ho11y container

To be able to look up logs of your workloads, you need to have a strategy or solution in place that is able to collect logs from the various pods and store and them in a central place. Let's have a look at a concrete setup, assuming you're using Amazon EKS with AWS Fargate in the data plane. In this case, the logging (http://mng.bz/D4Gy) setup is reduced to using a Kubernetes ConfigMap resource as an in interface to configure Fluent Bit as a log router that ships logs, for example, to CloudWatch. This would look as follows (output shortened):

Gets the content of the resource aws-logging and renders it as YAML

```
$ kubectl -n aws-observability \
          get configmap/aws-logging -o yaml
apiVersion: v1
kind: ConfigMap
data:
  output.conf: |
    [OUTPUT]
        Name cloudwatch_logs
        Match *
        region eu-west-1
        log_group_name ho11y-fluentbit
        log_stream_prefix fargate-
...
```

The Fluent Bit configuration to route application logs to CloudWatch

Specifies the AWS region to use

Specifies the log group name to send the logs to (will be created if it doesn't exist)

Adds fargate- to the beginning of each line (for easier filtering)

To consume the logs from the command line, you could, for example, use the `aws logs get-log-events --log-group-name ho11y-fluentbit …` command or the AWS console, as depicted in figure 3.3.

There are many options available to route and consume Kubernetes logs, such as the popular end-to-end setup using Elasticsearch and Kibana (http://mng.bz/lWKj) or the OpenSearch-based variant, as well as newcomers like Grafana Loki, which we will cover in detail in chapter 5.

In the Kubernetes control plane, as shown in figure 3.2, there are several components, such as the API server or etcd, that produce logs. Another source of logs in the Kubernetes control plane is the node-level supervisor (kubelet), usually available via